APAS-VERUS: AI Paired Proof Engineering Techniques and Experience

- Brian G. Milnes briangmilnes@gmail.com

- Experience, Results and Techniques in building
- Algorithms Parallel and Sequential by Acar and Blelloch
- in Rust
- and then proving it in Verus.
- Or **What Can We Prove Today and for How Much?**
- https://github.com/briangmilnes/{APAS-VERUS,APAS-AI,rusticate,veracity}

Outline of the talk

- Its a pleasure to speak here at CMU CS, my intellectual home.
- Background
- Algorithms Parallel and Sequential (APAS)
- Rust - The Good, The Bad and the Ugly
- APAS-AI - AI Paired Programming APAS in Rust
- Rusticate - sending Python back to the family estate
- Verus - Proving Rust

Outline of the talk

- APAS-VERUS - AI Paired Proving APAS in Verus
- Veracity - Software Engineering AI Paired Proving
- Software Engineering in AI Paired Proving
- CPR$ - how to measure costs and mine
- The AI Paired Programming Interfaces
- The Internet Apocalypse
- Review of the Talk
- Questions

Background

- B.Sc. Applied Math (Computer Science), Carnegie Mellon
- 2 years of AI PL development (Carnegie Group), my first startup.
- 7 years of (symbolic) AI Research (Soar Group, CMU CS)
- 2 years of PL research (Fox Group, CMU CS) aimed at making type safe languages work in systems and networking.
- Founding member of Lycos - systems, performance, ran operations
- Early Amazon - systems, performance, operations
- M.Sc. Computer Science, University of Washington
- Early Zillow - 6th engineer, ran operations, systems, performance

- A general love of proving programs with Rocq, F* and now Verus.

Algorithms Parallel and Sequential (APAS)

- I chose to implement: Algorithms Parallel and Sequential (APAS) in Verus.
- Umut Acar and Guy Blelloch 2022
  - 121 algorithms!
  - 81 of which can be parallel.
  - 740 concepts.
- Not only is it a great textbook but it proved with very few changes needed!
- Thank you Umut and Guy!

Rust - The Good

- 20 year old PL
- Fast compilation, code.
- Industrial acceptance: AWS, Google, Huawei, Microsoft, and Mozilla.
- Linux Kernel now uses it!
- Linear typing plus borrowing! No GC!
- Clear mutability.
- Slicing! with ownership.
- Their 'cargo' package system works rather well.
- No objects for modularity! Good.

Rust - The Bad

- No GC! Circular structures require GC and why not have it!
- No regions, just lifetime with end of scope deallocation which is slower.
- Translating C to Rust is hard to get it into linear logic + borrowing.
- However, you can (and I have) rewritten algorithms with a free list (that are right).
- Macros, with typing checked at use, which is not so good.
- No objects: it's great for what it was invented for: modeling the real world and interfaces by Ivan Sutherland in 1962! (sorry Bob!).
- Rust terminology is random.

Rust - Typeclasses are a weak module

- You get basic module with pub/pub(crate) and no-pub.
- You get typeclasses which MUST be implemented at type.
- The Rustaceans have decided that unless their module implements a typeclass at multiple types, they won't use it.
- So reading Rust is every bit as scattered as reading C.

- Verus is now doing this also, but I ABUSE the notation to put all the specs together in APAS-VERUS for readability.
- You will pine for ML modules!

Rust - The Ugly — Equality and Ordering

Property	PartialEq	Eq	PartialOrd	Ord
Reflexive	NO!	req	NO! on leq	yes
Symmetric	req	req		
Transitive	req	req	req	req
Antisymmetric			req	req
Total				req
Consistent w/ ==				

Rust - The Ugly

- Clone
  - fn clone(&self) -> Self
  - Informal contract: returns a value equal to *self
  - No hard language enforcement — you can implement a "clone" that returns something different , but it violates the convention.
- Copy: Clone
- Copy is a subtrait of Clone — every Copy type must implement Clone
- Hard contract: clone() must be equivalent to a bitwise copy, i.e., clone() == *self
- This is documented in std: "if T: Copy, T::clone(&x) must be equivalent to copying x"
- What you really want here is to detangle these.

APAS-AI - AI Paired Programming APAS in Rust

- APAS-AI is a nearly complete, idiomatic Rust implementation of the algorithms from Acar and Blelloch.
- Sequential and parallel variants throughout.
- Timeline: 347 commits over 88 calendar days (Aug–Oct 2025).
- 59 days of active development; 8 residual commits in November.
- I knew no AI paired programming when I started.
- I knew no Rust when I started.
- My AI had to teach it to me, which was harder than I thought as of the 94 terms used in the Rust language and docs, only 4 are real PL terms!

APAS-AI - AI Paired Programming APAS in Rust

- Scale:
  - 42 chapters
  - 238 source files
  - 45,485 source LOC.
- Tests:
  - 246 files
  - 55,223 LOC
  - 3,923 test functions
  - 1.2× the source code size which is heavy.
  - But tests are cheap now.
- Benchmarks: 171 files, 13,890 LOC, 360 benchmark functions

Rusticate - sending Python back to the family estate

- Built because Python + regex cannot reliably parse Rust.
- Webster's definition of rusticate:
  - 1. To go to or live in the country.
  - 2. (British) To suspend a student from a university, especially Oxford or Cambridge, as a disciplinary punishment.
- Rusticate is a suite of 89 tools, most deprecated, for analyzing and transforming rust codebases.
- review tools, fix tools, metrics, and migration aids.
- The string hacking detector is the foremost valuable tool to fix any and all transformation of Rust.

Verus: Verified Rust

- Verus is a tool for formally verifying Rust, developed at MSFT Research, VMware Research and Carnegie Mellon University.
- Team: Andrea Lattuada, Travis Hance, Chris Hawblitzel, Jay Lorch, Matthias Brun, Chanhee Park, Yi Zhou, Jon Howell, Bryan Parno.
- Goals: bring machine-checked proof to systems software written in Rust.
- Goals: tight integration with the programming language.
- Goals: easier faster proof.

Key Verus Publications

- "Verus: Verifying Rust Programs using Linear Ghost Types" Lattuada, Hance, Cho, Brun, Subasinghe, Zhou, Howell, Parno, Hawblitzel
- "Verus: A Practical Foundation for Systems Verification" Lattuada, Hance,

Bosamiya, Brun, Cho, LeBlanc, Srinivasan, Achermann, Chajed, Hawblitzel, Howell, Lorch, Padon, Parno
- "Verifying Concurrent Systems Code" Travis Hance — PhD Thesis, Carnegie Mellon University, 2024

Verus

- Specifications are written in a pure math sublang: First order logic.
- spec functions, forall/exists, arithmetic, sets, sequences.
- Undecidable.
- Z3 handles linear arithmetic, arrays, and quantified formulas, but quantifier instantiation requires explicit trigger annotations.
- But there is also a faster linear arithmetic solver, Singular.
- Wraps existing Rust code:
  - spec / proof / requires / ensures on fns
- Ships the Rust binaries directly.
- Linear Logic + Borrowing from the Rust type system, which rustc checks.
- The TCB then is Verus+Z3+RustC+STD, which is BIG.

Views and the Libraries

- A View maps an executable Rust type to a mathematical ghost type: "Vec views as Seq", "HashSet views as Set".
- Specs are written over the view; exec code manipulates the real type.
- vstd is Verus's standard library — specs for Vec, Seq, Set, Map, Multiset, arithmetic, common lemmas, Fns …
- Ghost types live only in the verifier — they have no runtime cost and no runtime representation.

UnDirGraphStEph — Data Struct + View

```
#[verifier::reject_recursive_types(V)]
pub struct UnDirGraphStEph<V: StT + Hash> {
    pub V: SetStEph<V>,
    pub E: SetStEph<Edge<V>>,
}

impl<V: StT + Hash> View for UnDirGraphStEph<V> {
    type V = GraphView<<V as View>::V>;
    open spec fn view(&self) -> Self::V {
        GraphView { V: self.V@, A: self.E@ }
    }
}
```

UnDirGraphStEph — Trait

```
pub trait UnDirGraphStEphTrait<V: StT + Hash>:
    View<V = GraphView<<V as View>::V>> + Sized
```

```
{
    open spec fn spec_neighborhood(&self, v: V::V) -> Set<V::V>
        recommends spec_graphview_wf(self@), self@.V.contains(v)
    { Set::new(|w| self@.A.contains((v,w)) || self@.A.contains((w,v))) }

    fn neighborhood(&self, v: &V) -> (nbrs: SetStEph<V>)
        requires spec_graphview_wf(self@), self@.V.contains(v@), ...
        ensures  nbrs@ == self.spec_neighborhood(v@), nbrs@ <= self@.V;
}
```

UnDirGraphStEph — Impl: `neighborhood` exec body

```
fn neighborhood(&self, v: &V) -> (nbrs: SetStEph<V>) {
    let mut nbrs: SetStEph<V> = SetStEph::empty();
    let mut it = self.E.iter();
    let ghost edges_seq = it@.1;
    loop
        invariant nbrs@ == Set::new(|w| exists |i|
            0 <= i < it@.0 && /* edges_seq[i] hits v on either side */ ...),
        decreases edges_seq.len() - it@.0,
    {
        match it.next() {
            None    => { proof { /* set-equality lemmas */ } return nbrs; }
            Some(e) => { /* if feq(&e.0,v) insert e.1; sym. */ ... }
        }
    }
}
```

Wrapping Rust — Declaring an external type

- Four specification constructs are used to give specs to Rust stdlib.
- A proxy struct that introduces a spec for a foreign type.
- The proxy struct name is conventionally ExTypeName.

```
#[verifier::external_type_specification]
pub struct ExVec<T>(Vec<T>);
```

Wrapping Rust — Declaring an external function/method

- A proxy function with the same signature as the foreign function, carrying the requires/ensures.

```
#[verifier::external_fn_specification]
pub fn ex_vec_push<T>(v: &mut Vec<T>, value: T)
    requires v@.len() < usize::MAX,
    ensures  v@ == old(v)@.push(value),
{ v.push(value) }
```

Wrapping Rust — Declaring an external function/method

- Add a View and specs to a foreign type.

```
#[verifier::external_type_specification]
pub struct ExHashMap<K, V>(HashMap<K, V>);
impl<K,V> View for ExHashMap<K,V> {
     type V = Map<K::V, V::V>;
     spec fn view(&self) -> Map<K::V, V::V>;}
```

Wrapping Rust — external_trait_specification

- Adds a spec to a foreign trait without modifying it:

```
#[verifier::external_trait_specification]
pub trait ExClone: Sized {
    type ExternalTraitSpecificationFor: core::clone::Clone;
    fn clone(&self) -> Self;
}
```

- The proxy trait name is `Ex<TraitName>` by convention.
- Add `requires`/`ensures` to the method to give it a full contract.
- Limitation: no generics.

Tokenized State Machines — Hance, CMU 2024

- Problem: Rust's ownership types handle sequential aliasing well but cannot express distributed protocol state across threads.
- Answer: A Tokenized State Machine defines protocol state as fields with sharding strategies (variable, map, count, storage_option…).
- Transitions and an inductive invariant are proved once, globally.
- Verus auto-generates ghost token types and exchange functions

Tokenized State Machines — Hance, CMU 2024

- So client code manipulates local tokens, not global state.
- RwLock in vstd is implemented via a tokenized state machine
- The lock's internal protocol (unlocked / read-locked / write-locked, reader count) is the TSM.
- RwLockPredicate is the (single) invariant the user supplies.

Tokenized State Machines — Hance, CMU 2024

- Hance et al. "Sharding the State Machine" — OSDI 2023 (primary TSM paper)
- Hance, Howell, Padon, Parno. "Leaf" — OOPSLA 2023 (storage protocols)
- Hance. PhD Thesis, CMU-CS-CS-24-146, 2024 (full formal treatment)

Verus: How Fast Verus Is Moving

- 4,225 commits since March 2021 — 5 years, still accelerating
- Commits per year:
- 2021: 474 2022: 906 2023: 992
- 2024: 745 2025: 761 2026: 347 (thru April)
- Rolling releases: 396 tagged — roughly one every 4 days
- 59 stable releases!
- Excellent team and work. I have had only a few crashes, which I could work around.

APAS-VERUS - AI Paired Proving APAS in Verus

- Goal: formally verify all algorithms in Acar and Blelloch
  - Every algorithm gets a machine-checked proof
  - no admitted lemmas,
  - no hand-waving in production code.
- 44 chapters, 262 algorithm files, upto 4 variants per algorithm: StEph (sequential mutable), StPer (sequential persistent), MtEph (parallel mutable), MtPer (parallel persistent).
- Minimal use of Rust std.
- But as you'll see I had to write some axioms and admit a class of functions.

APAS-VERUS - AI Paired Proving APAS in Verus

- 26 vstdplus library modules, mostly making Views.
- 29 standards documents encoding project proof conventions,
- Verification is the primary goal.
- Runtime tests (RTT) and proof-time tests (PTT) are secondary, but still terribly useful.
- They both still catch errors and instruct the AI.

APAS-VERUS — Quantitatives

- Scale: 44 chapters, 262 files, 186,223 src LOC (not counting comments).
- With vstdplus, standards, RTT, PTT: 275,014 total LOC.
- Built in 160 days, 2,596 commits, 8 agents, 281 agent-round reports.
- Verification: 5,674 verified proof obligations, 0 errors.

APAS-VERUS: Full Validation Cost (2026-04-12)

- Elapsed: 210s
- rust_verify RSS: 10,278 MB (~10 GB)
- Z3 RSS: 6,874 MB (~6.7 GB)
- rust_verify CPU: 216s
- Z3 CPU: 265s
- But I have had Z3 jump up to as much as 28 GB when I write bad proofs.
- Somewhere in it I suspect there is a novel formal verification of some algorithm.

APAS-VERUS — Quantitatives

- Runtime tests: 3,776 pass in 21 s.
- Verus has a nice proof-time test harness, so I pulled it out:
  - 221 pass in 259 s.
  - I used it mostly to continuously track that my iterators prove and continue to prove with verus changes and my specifications.
  - These caught dozens of problems that would have been conflated with algorithms loops.
- Benchmarks in 42s.

APAS-VERUS — Quantitatives

- Holes: started at 238 (R20), now 0!
- Largest chapter is the forest: Chap37 - AVL trees, BST variants - 20,319 src LOC.
- 2 × more source code to verify than APAS-AI needed to implement.
- Start: 2025-11-03
- End : 2026-04-12
- Duration: 150 calendar days, roughly (more later)

APAS-VERUS -

- Spec 32,868 (21%)
- Proof 42,251 (27%)

- Exec 67,883 (44%)
- Rust 12,206 (8%) plain Rust (outside Verus!)
- The "rust" 8% is code outside Verus! — Debug impls, macros, cfg, etc.
- All proof to exec: 75,119 / 67,883 = 110%.

APAS-VERUS: The Pain Points

- When I started Verus, iterators for collections took quite some time.
- Generics and Equality was the second big pain point.
- I still have full equality axioms for generic types.
- Ordering was and is still difficult, it made my UnionFind consume up to 28GB in Z3.
- Verus is so fast, even with bloated AI proofs, I didn't profile much until I was in the last few chapters.
- I simply made validate isolate by chapters.

APAS-VERUS: The Pain Points

- Closures took a good bit of work.
- Applying them in map/reduce and so on took a while.
- Rust has FnOnce < FnMut < Fn, but no real FPure.
- You can const fn and the compiler will check it but then you can't say: `fn map(const fn F)`
- Verus Ghost functions allows good validation but purity would be simpler.

AutoCLRS

- AutoCLRS is Swamy et al. and AIs implementation: "Introduction to Algorithms, 4th ed" 2022
- by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
- in Pulse.
- RISE MSR blog (2026-03-06) says the initial 10K lines came "very quickly" and then "about a month of nudging" to reach 100K LOC.
- And now seems to be about 130K LOC.
- Nikhil Swamy with thanks to Gabriel Ebner, Lef Ioannidis, Guido Martinez, Matthai Philipose and Tahina Ramananandro.

AutoCLRS

- They definitely had some tool advantages in terms of incremental proofs through a server.
- That's another verus pain point but not too bad.

- Plus, they knew F* and Pulse to start!
- And they did a formal specification of algorithmic complexity!

Veracity- Software Engineering AI Paired Proving

- Veracity is a suite of 22+ tools for analyzing, reviewing, and fixing Verus codebases.
- Review tools:
  - proof holes (assume, external_body, admit),
  - style enforcement (21 rules, auto-reorder),
  - with spec strength classification fed to AI,
  - veracity-count-loc (spec/proof/exec breakdown),
  - chapter-cleanliness-status (clean vs. holed chapter summary vs blocked by),
  - string-hacking detector, function inventory, etc.
- One of the best is veracity-minimize-proofs

Veracity- Software Engineering AI Paired Proving

- All tools are AST-aware (ra_ap_syntax / Verus_syn).
- A string-hacking detector flags usage of string manipulation instead of AST work.
- And when bugs appeared they were mostly string hacking.
- Because no matter what I said to my AIs they LOVE string hacking shortcuts.
- Heck, I had to run the string hacking detector on the string hacking detector.

Veracity- Software Engineering AI Paired Proving

- Search: veracity-search — type directed search over vstd
- VERUS by type signature, finding lemmas before writing new ones.
- "Specifications as Search Keys for Software Libraries" Eugene J. Rollins and Jeannette M. Wing
- Written for my sins of asking why does vstd not have X, when it did!
- Even more useful for my AIs's seriously disturbing sinning.
- This allowed me to download ALL known Verus (git VerusCodebases) and have my AI search them in 1.2 seconds!
- Does F*/Pulse have one yet?
- It only took about a day.

APAS-VERUS - Complexity

- APAS states complexity and informally proves many of them for some algorithms.
- I had to build a tool to get the right ones in the code at the right place.
- My single threaded implementations often don't match the textbook intentionally.
- Then I wrote a programmatic tool to find and list mismatches.
- Then I had Claude Opus do it's analysis and compare every function with the textbook's.
- This found 16 faults in parallel algorithms.

APAS-VERUS: Verified Iteration - Pain Point

- Iteration in Rust is rather complex.
- 70 functions on iterator but only 7 functions cover the 90% case.
- And in Verus it's a bit more complex and takes some time to learn.
- 10 components required per collection (all inside Verus!)
- 6 verified loop patterns per collection
- {loop, for} X {borrow iter, borrow into, consume}
- Iterators requires one assume!

APAS-VERUS: Verified Iteration

- Verus forbids adding requires on external trait impls (std::iter::Iterator)
- Hand-rolled iterators need assume(iter_invariant(self)) in next()
- Everything but that one assume is fully proved
- 44 collections implemented; all carry verified iterators
- Verus has proof time tests inside, I freed them to run in APAS-VERUS.
- This was critical to get iterative loops to prove over my collections ADTs.
- You can prove full iterators in your own copy of the traits with no assume and I have.

APAS-VERUS: Experiments

- Agents often say "Verus Can't Do That"
- I said "Make an experiment!"
- Quantitatives:
  - 168 experiment files
  - 21,476 lines of code (not counted in the totals)
- Topics span: Clone, Arc, RwLock, TSM, closures, iterators, generics, float, bitvector, PartialEq, Copy, async, hash tables, parallel algorithms, ghost types, Send/Sync, collect, and sorting.

APAS-VERUS: Experiments

- Results:
- 107 files: SUCCEEDS / VERIFIES — pattern adopted into codebase
- 61 files: FAILS — Verus limitation documented, workaround noted
- Notable successes that unlocked chapters:
  - TSM/RwLock layer pattern — unlocked all Mt modules,
  - Named closure ensures through ParaPair — unlocked fork-join,
  - Ghost struct Send/Sync — unlocked AVLTreeSetsMtEph,
  - Tree module style — unlocked Kruskal, Prim, UnionFind.

APAS-VERUS Standards

- I finally built a set of coding standards in Verus rust files and in comments.
- Agents read all standards before every task (~6,200 lines, ~54K tokens)
- Violations are mostly AI checked except where an extensive code styling can get things.
- Quantitatives:
  - 29 standard files
  - 6,911 lines total
- Doing this earlier would have really sped things up.
- My CLAUDE.md would just not do enough even with 50KB and 13K tokens.

APAS-VERUS RULEs

- Question what are your favorite AI rules?
- Mine are:
  - Don't Over Think,
  - DISCUSS,
  - Don't jump ahead,
  - go step by step,
  - PBOGH: Prove Big or Go HOME!
- You just have to tell the agents keep on proving!
- Are you building big CLAUDE.md?
- Or Cursor Rules?

Veracity: AIs Write Redundant Proofs

- AI proof agents produce many correct but bloated proofs
  - redundant asserts, unnecessary proof blocks
- They verify, but they waste solver budget on every subsequent run.
- So I wrote a proof minimizer: veracity-minimize-proofs.

- It tests each assert and proof block individually: removes it, re-verify, comment it out if it is not needed and it does not increase time or memory.
- Result across APAS-VERUS: 22 asserts and 33 proof blocks removed in 105 minutes of wall time. 55 redundant proof statements eliminated, ~2 minutes of minimizer time per removal.

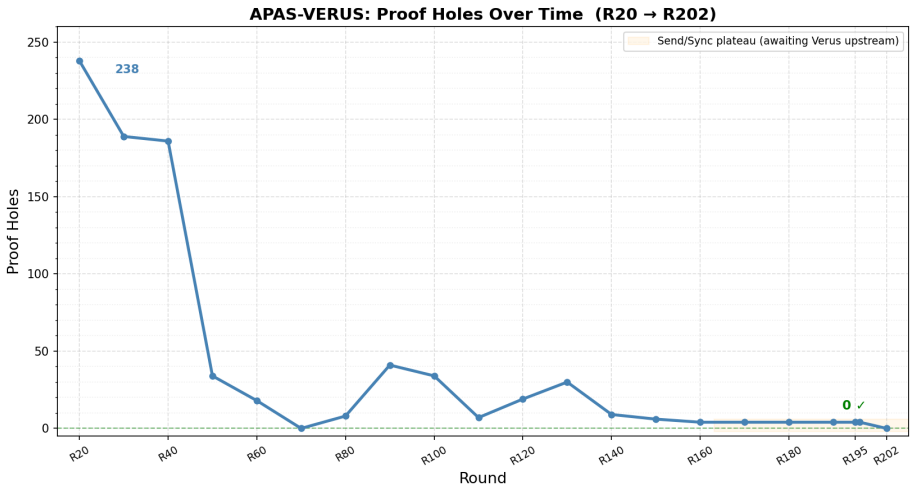
Veracity: AIs Write Redundant Proofs

- One assert in Chap43 OrderedTableMtEph saved:
  - 104 s of Z3 CPU
  - up to 89 MB of Z3 RSS per verify run.
- Eight removals in that one file: Z3 RSS dropped by 57%.
- 105 minutes of running the minimizer bought many hours of validation drop.
- This might be novel. Anyone know of anything except Isabelle's sledgehammer?

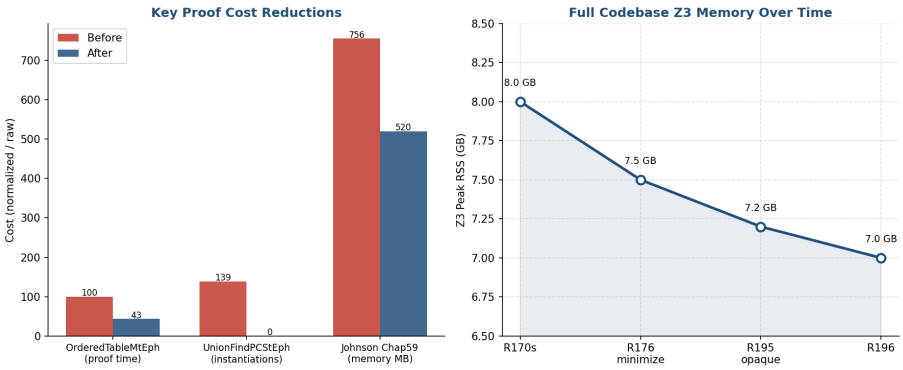
Veracity Annotations

- I ended up having to have my tools work mostly in comments.
- I added accept(P) to mark assumes I allowed, almost all eq/partialeq/clone.
- This can be simplified with some Verus language syntax, but then the AIs are not trained on the symbols.
- 6,081 // Veracity: NEEDED assert
- 4,681 // Veracity: NEEDED proof block
- 1,502 // Veracity: NEEDED assert (speed)
- 245 // Veracity: NEEDED proof block (speed)—
- Total annotations: 14,140

Proof Holes Over Time — R20 to R201



Proof Time and Memory — Key Reductions



Proof Time and Memory — Key Reductions

- Five techiniques were used to optimize
  - minimize-proofs
  - profiling
  - splitting specs and applying them just where they are needed
  - opaque - to hide a definition within the module
  - private spec - to hide a definition across modules
- OrderedTableMtEph: −57% proof time after minimize-proofs (R176).
- UnionFindPCStEph: 139K Z3 instantiations → 0 after opaque pattern used in a required module.

- Johnson Chap59: 756 MB → 520 MB (−31%) memory reduction.

CPR$ — Three Numbers and One Ratio

#	Sym-bol	Name	Measures
1	C	Cost of Code	$ to produce executable code
2	P	Cost of Proof	$ to produce specs, contracts, and proofs
3	C+P	Total	Full bill for the verified artifact
4	R	Review Ratio	Fraction of deliverable a proofgrammer must read (LOC0R excluded)

Inputs & Computation

- total_hours = person_days × avg_hrs_per_day
- programmer_cost = total_hours × (programmer_costs / 1,760)
- AI_cost = total_hours × (ai_costs / 1,760)
- LoEC_share = LoEC / total_LOC
- C = total_cost × LoEC_share
- P = total_cost × (1 − LoEC_share)
- R = LOPC2R / (LOPC2R + LOC0R)

Worked Example — Inputs

- **Programmer rate**: $375,000/yr (senior + 50% loading)
- **Real AI spend**: < $7,000 across the whole effort
- **Derived `--ai-costs`**: $4.886/hr × 1,760 = **$8,599/yr**
- **AI split by task-hours**: 21.6% APAS-AI ($1,512) / 78.4% APAS-VERUS ($5,488) | # | Project | Hours | LOC | LoEC | LOPC2R | LOC0R | |:–|:————————-|———:|———:|——-:|——-:|——-:| | 1 | APAS-AI + Rusticate | 309.6 | 31,751 | 31,751 | — | — | | 2 | APAS-VERUS + Veracity | 1,123.4 | 166,401 | 56,549 | 47,828 | 95,908 | | 3 | Combined | 1,432.96 | 198,152 | 88,300 | 47,828 | 95,908 |

CPR$ — Combined Result

#	Quantity	Value
1	C — Cost of Code	**$150,723**
2	P — Cost of Proof	**$161,594**

#	Quantity	Value
3	C + P — Total	**$312,317**
4	R — Review Ratio	**33.3%**
5	$ / verified deliverable line	**$1.877**
6	C / KLOEC	**$2.665**
7	P / KLOPC2R	**$3.378**
8	P / KLOP	**$6.305**

Head-to-Head — seL4 (2009) vs APAS-VERUS

#	Quantity	seL4 (pre-AI)	APAS-VERUS (AI-paired)	Ratio
1	Person-years	22	0.64	seL4 ~34×
2	Hours	45,760	1,123	seL4 ~41×
3	KLOE	10	~57	Verus ~5.7×
4	KLOP	480	~110	seL4 ~4.4×
5	KLOP / KLOE	48	~1.9	seL4 ~**25×**
6	Klines / hour	0.0107	0.148	Verus ~14×
7	$ / KLOE	$825,000	~$4,300	seL4 ~**192×**
8	$ / KLOP	$17,188	~$2,225	seL4 ~7.7×
9	C + P total	$8,250,000	$244,840	seL4 ~**33.7×**

Rusticate + Veracity allow quantitative software engineering

- I downloaded the 1036 most downloaded Rust projects, 3636 crates.
- I threw out three that wrapped std for asynchrony.
- I did a greedy set cover analysis of what Rust uses.
- And another of what verus wraps.
- This is a golden age for quantitative software engineering.
- It took just two days of person time.
- And I never read a line of code in the tools. Just check the output and have the AI write a lot of tests.

Rusticate + Veracity : What Rust Cargos use in std.

- Top 1000 projects, 3636 crates.
- 19 data types fully support 90%.
- 48 data types fully support 100%.

- 69 modules fully support 95%.
- 79 modules fully support 99%.
- 1121 methods fully support 95%.
- 1733 methods fully support 99%.
- 14,317 total fn definitions in std/core/alloc in Rust
- 4,965 pub fn definitions in std/core/alloc in Rust
- But how many of those private functions do we need? I don't know.

Compare Rust STD to APAS-VERUS?

- APAS-VERUS: -6,401 exec functions total, -4,911 with proofs
- But APAS-VERUS has {Mt,St}x{Per,Eph}
- So it is much more like 2000 distinct functions.
- And I wrote this in 160 days while learning Verus.
- At least 30 of those days were understanding and working around pain points.

Rusticate + Veracity: What Verus Wraps

- As of 2026 14 April
- How many Rust Data Types does Verus wrap?
  - 29 types currently wrapped.
- How many Rust Traits does Verus wrap?
  - 147 traits in vstd.
- How many total Rust Methods does Verus wrap?
  - 154 methods currently wrapped.
- A small but growing percentage.

Software Engineering in AI Paired Proving

- Software engineering in the AI age is much like managing programmers.
- You can't possibly read all their code.
- So you use:
  - linters both programmatic and AI
  - stylers both programmatic and AI
  - AI reviews directed by programmatic tools.

Software Engineering in AI Paired Proving

- Starting with a textbook that has a ton of good prose is a huge plus.
- I did discover a few things about the textbook.
- It needs more on Scheduling. I only built Help-First scheduling.

- It used (K,V) in tree sets as mappings, but I had to make this explicit. Particularly with an Ordered Key Mapping.
- With this Ordered Tables, which are not too complex, were much easier to prove.
- APAS's discussion of Union Find was too thin. Path compression was very difficult.
- I had to start with Nikhil Swammy et. al's AlgoCLRS's implemention and proofs.

Software Engineering in AI Paired Proving

- I use git work trees for proof.
- The good news is git is powerful and the agents understand it.
- The bad news is the agents can indeed start using advanced features and get things wrong.
- I work with between 1 and 9 agents.
- One on veracity.
- One orchestrator, writing plans, prompts and controlling merges.
- 1-8 on branches each working on different independent plans.
- I am not nearly comfortable enough with agent proof to let them do subagents without observing.
- Cursor was much better at interruption to guide things.
- Claude is getting better at it.

Software Engineering in AI Paired Proving- Models

- The agents I used are: various Google, OpenAI and Anthropic.
- A google agent once took 12 minutes to 'echo HI' to test it's terminal.
- Anthropics models are signficantly better than OpenAI.
- And Claude Code's cost is the least so far.
- They are said in the press to be spending $5000 per month per user.
- They are buying market share (what happened to Amazon's buying market share phase?).
- And they are buying coding interactions.
- But both vendors are playing leap frog.

Software Engineering in AI Paired Proving- Problems

- They are inconsistent.
- They lie.
- They cheat.

- They say I can't prove that!
- They err.
- But mostly they are just forgetful.
- My attempts to give them coding standard checklists, ala Watts Humphrey and the PSP failed.
- Sometimes they would say "The checklist was followed" and then admit "I lied."

Software Engineering in AI Paired Proving- Problems

- They are getting better, very few code hallucinations.
- If you tell them you are Dr. FunkenProof and they are Igor, they hallucinate whole modules.
- Just a few months ago only Claude could sweep a codebase decently.
- Now both Claude and OpenAI can.
- And now they can prove like heck.
- 10x what I can do.
- Claude wants $25 per code review now, probably because they can and it's expensive if they're using pure LLMs.

Software Engineering in AI Paired Proving- Problems

- What is the limiting factor now? **Code Review!**
- But really **Spec Review** when you prove.
- What can we do to simplify code review?
- 1. Modularity - traits/impls in Verus.
- 2. TOC - This organizes the boilerplate to make reading easier.
- 3. Proof - I really just read the specs. If they're right the code is right!
- 4. Tests - are hugely useful when you don't trust your coding team.
- 5. Formatting - rust formatting has been adopted in Verus. It is very low density. I built a minimizing formatter to cut down on my working memory load while reviewing. F* is much tighter.

The AI Paired Programming Interfaces

- What you want is pretty obvious with experience: TEXT still rules.
- You want one window with your core interaction. The agent tells you what it is doing in it.
- You want one to watch your compile, tests and scripts.
- You want one window to have the LLM show you it's thinking and allow you to question and change it.

- I learned a ton from watching agents think.
- I suspect they are hiding most of their thinking now to prevent them from being used to train new AIs.
- And you want it not to delete your previous conversation on token compression.

Mythos Audit Rust/Firefox

#	Metric	Rust (Compiler/Std)	Firefox (Entire Codebase)
1	Codebase Size	~4,500 KLOC	~40,000 KLOC
2	Mythos Bugs Found	189	271
3	Vulnerability Density	0.0420 bugs/KLOC	0.0068 bugs/KLOC
4	Ratio	**6.18**	1

The Internet Apocalypse

- I asked Claude Code to check its switches on startup for me, at the End of Jan 2026.
- It immediately started disassembling itself.
- Scared the heck out of me.
- I told it to read it's docs.
- I have 100% belief that these LLMs are moving faster than computer security.
- It's verify or be hacked at this point.

The Internet Apocalypse

- Where should we focus our limited, but quickly growing, proving power?
- In theory 70% of intrusions are solved by type safe languages.
- I suspect that this just ignores way too much of the human aspect.
- I have never liked a single estimation paper's methods.
- One way to help us focus our new-found superpowers is to survey KLOC type-unsafe x Interfaces x Running Services x $ Value.
- Which Amazon and Microsoft can probably measure.

Future Work

- GRASE (Medical Acronym) -
  - Generally Recognized as Safe and Effective
- **GRASE-**
  - **Git Recording Agentic Software Engineering**

- The GRASE project's goal is to make statistical process software optimization for agentic coding:
  - **Generally Regarded As Simple and Effective.**
- A Watt's Humphrey style PSP data collection done essentially effortlessly by the agents in the background.

Review of the Talk

- Ah let's just get to the questions!
- I have a ton of them for you! and I hope you have many for me.

Extra Slides

AI Paired Programming Size: Quick Review

- APAS-AI Lines of Code
  - Source: 31,751
  - Tests: 55,223
  - Benches: 13,890
  - Total code: 100,864
- Rusticate Lines of Code
  - Source: 29,582
  - Tests: 12,890
  - Total code: 42,472

AI Paired Proving Size: Quick Review

- APAS-VERUS Lines of Code
  - Source: 229,319
  - Tests (RTT): 58,503
  - PTTs (rust_verify_test): 15,938
  - Benches: 8,027
  - Total code: 311,787
- Veracity Lines of Code
  - Source: 65,189
  - Tests: 2,851
  - Total code: 68,040

Blood on the Road - Open Source =

- Open Source is now Open Copy.
- with reports of 1/2 of AI time
- and with the US Government totally undermining copyright law.

Blood on the Road - Open Repositories =

- Open repositories are now weakly locked doors.
- Agents can and have been used to inject invisble UNICODE attacks.
- Agents can now pass the Turing test, and more easily the programmer Turing test.
- They'll submit code pretending to be human.
- They'll steal credentials and submit code.
- LiteLLM is the most atrocious case lately.

Blood on the Road - Key Services

- Authentication is going to be attacked brutally.
- Network services are going to be hacked left and right.
- Configurations are going to be heavily attacked.
  - Using text files for these is already error prone.

Blood on the Road - Open Binaries =

- Open binaries are next.
- They are going to be disassembled and recontructed.
- They are going to be rewritten in real time.
- Key solutions to this are:
- Binary packages from all repositories.
- Cryptographically signed.
- Installation protected in the OS.
- And protected from reading by the OS.

Questions - F* and Pulse

- Are F* modules really in use? If not, why?
- Got Functors?
- Are F* modules working with proof smoothly? Took me about three days to get a proving model (Treap) with modules.
- Does F* have a typed library search yet?
- Are typeclasses being heavily used?
- How fast is validation now?
- Nik you have any more statistics on AutoCLRS?

Questions

- Did the PL community overly complicate things with higher-order types?
- What can you folks say about MSFT adoption of proven code?

- What are the biggest F* verified systems?
- What are the biggest Pulse verified sytems?
- Any quantitatives? Person days? AI days?
- Should I quit Verus and go learn Pulse?

Questions

- Are you folks still using Make? That's how I got started on this.
- Many programmers and worse, managers, complain about the programmer time to switch languages. Are you finding that you can switch languages with your AIs explaining things to you?
- Can we prove Rust's (awful) std with Verus?
- Can we prove a compiler with Verus and EPR?
- Can we prove a TIL (Tarditi, Morrisett, Harper) like compiler?
- Can we verify a comp cert like compiler?
- Can we prove an OS in Verus?

Questions - AI LLM Agents

- How fast is this moving?
- How much context do we need?
- How much speed do we have? T/S down and up?
- How much speed do we need?
- How much speed will we get?
- When it's 10 times faster how will we use it?
- When will this really cost the user what it costs the provider?
- When will it get even better intermediate term memory?

Cloud Security Alliance

- The cloud security alliance has some sober recommendations:
- The "AI Vulnerability Storm": Building a "Mythosready" Security Program
- https://labs.cloudsecurityalliance.org/wp-content/uploads/2026/04/mythos-readyv91.pdf
- Use LLM-based vulnerability discovery and remediation capabilities.
- Update risk metrics.
- We cannot outwork machine-speed threats. Re-prioritize, automate, and prepare for burnout.